

A importância da divisão de tarefas no desenvolvimento de software e na engenharia de sistemas com Inteligência Artificial

Arthur Delfino Duarte

Bacharel em Engenharia Elétrica, com ênfase em Eletrônica e Telecomunicações.

arthur.d.duarte@live.com

Abstract—The division of tasks is a fundamental principle in software engineering and, more recently, in artificial intelligence systems based on large language models (LLMs). This article discusses the cognitive limitations of the human brain, the costs of context switching, and how specialization improves productivity and quality. It draws a parallel between traditional software teams and multi-agent AI architectures, demonstrating that dividing complex problems into smaller, specialized tasks produces more reliable, maintainable, and scalable systems.

Index Terms—Multi-agent systems, LLM, software engineering, task division, cognitive load, context switching.

Resumo—A divisão de tarefas é um princípio fundamental na engenharia de software e, mais recentemente, em sistemas de inteligência artificial baseados em grandes modelos de linguagem (LLMs). Este artigo discute as limitações cognitivas do cérebro humano, os custos da troca de contexto e como a especialização melhora a produtividade e a qualidade. Traça um paralelo entre equipes tradicionais de software e arquiteturas multiagente de IA, demonstrando que dividir problemas complexos em tarefas menores e especializadas produz sistemas mais confiáveis, sustentáveis e escaláveis.

Palavras-chave—Sistemas multiagente, LLM, engenharia de software, divisão de tarefas, carga cognitiva, troca de contexto.

I. INTRODUÇÃO

Existe uma razão pela qual praticamente todas as empresas de tecnologia do mundo trabalham com equipes especializadas em vez de esperar que uma única pessoa faça todo o trabalho. À primeira vista, pode parecer que um único desenvolvedor controla melhor o projeto, pois conhece todas as partes do sistema. Entretanto, conforme a complexidade cresce, essa estratégia se torna cada vez menos eficiente.

Curiosamente, exatamente o mesmo fenômeno acontece quando utilizamos modelos de linguagem (LLMs). Muitos usuários tentam construir um único prompt gigantesco para fazer tudo, quando, na realidade, sistemas modernos de IA são muito mais eficientes quando dividem

responsabilidades entre agentes especializados. O princípio é exatamente o mesmo: dividir problemas complexos em problemas menores produz sistemas mais rápidos, mais confiáveis e mais fáceis de manter.

Este artigo explora essa interseção entre engenharia de software e inteligência artificial, apresentando fundamentos teóricos sobre carga cognitiva, especialização e arquiteturas multiagente, com exemplos práticos de aplicação.

II. FUNDAMENTOS DA DIVISÃO DE TRABALHO

A. A Limitação Biológica do Cérebro Humano

O primeiro conceito importante é entender que ninguém consegue manter um projeto inteiro na memória de trabalho. Durante um desenvolvimento existem dezenas de atividades simultâneas: arquitetura, banco de dados, APIs, frontend, autenticação, segurança, UX, documentação, testes, deploy, monitoramento e otimização. Cada uma dessas áreas exige um tipo diferente de raciocínio.

Essa dificuldade não decorre de falta de habilidade, mas de limites cognitivos. A memória de trabalho humana consegue manter apenas uma quantidade limitada de informações simultaneamente. Pesquisas na área de produtividade em engenharia de software mostram que desenvolvedores consideram um dia produtivo justamente aquele em que conseguem trabalhar longos períodos sem interrupções significativas. Em um estudo com 379 desenvolvedores profissionais, interrupções e trocas frequentes de contexto apareceram como um dos principais fatores percebidos como prejudiciais à produtividade.

A complexidade não cresce de forma linear; ela cresce com as interações entre os componentes. Em um sistema pequeno, um desenvolvedor consegue manter praticamente todo o contexto na cabeça. À medida que

novas funcionalidades surgem, aumenta não apenas a quantidade de código, mas também o número de relações entre módulos, dependências e decisões arquiteturais.

B. Os Três Papéis Fundamentais

Mesmo quando apenas uma pessoa desenvolve um sistema, ela normalmente assume três papéis distintos. O primeiro é o Planejador, responsável por responder o que será construído, como será construído, qual arquitetura utilizar e quais tecnologias empregar. Seu foco é estratégico — ele não escreve código, ele pensa. O segundo é o Executor, que transforma o planejamento em código. Seu objetivo é produzir, e ele não deveria ficar redesenhando a arquitetura enquanto implementa. O terceiro é o Supervisor, que pergunta continuamente: está funcionando? Há bugs? O código está limpo? Está seguro? Está documentado? Os requisitos foram atendidos? Ele atua como revisor.

Quando uma única pessoa faz tudo, ela alterna continuamente entre esses três modos mentais. Isso gera fadiga cognitiva e reduz a qualidade do trabalho produzido.

C. O Custo Invisível da Troca de Contexto

Suponha um desenvolvedor implementando autenticação. Durante essa tarefa ele mantém mentalmente: fluxo de login, tokens, banco de dados, criptografia, sessões, APIs e requisitos do cliente. Subitamente surge um bug em outro módulo. Agora ele precisa descarregar todo esse contexto e carregar outro completamente diferente. Depois de resolver o bug, precisa reconstruir o estado mental anterior.

Cada troca de contexto exige que o cérebro reconstrua mentalmente o estado anterior. Isso custa tempo — muito tempo. Estudos em engenharia de software mostram que recuperar completamente o contexto após uma interrupção pode levar vários minutos, e interrupções frequentes reduzem significativamente a produtividade. Pesquisadores da área de interação humano-computador estudam esse fenômeno há décadas. Os resultados mostram que interrupções frequentes podem aumentar o estresse, elevar a carga mental, dificultar a retomada da tarefa original e reduzir a percepção de produtividade. Esses efeitos são especialmente relevantes em atividades criativas e analíticas, como programação, arquitetura e revisão de código.

D. O Conceito da Carga Cognitiva

Todo cérebro possui capacidade limitada. É útil imaginar uma mochila: cada tarefa adiciona peso. Arquitetura, banco de dados, frontend, Docker, segurança, documentação e testes — cada uma dessas áreas contribui para a carga mental total. Quando a mochila fica pesada, surgem esquecimentos, bugs, decisões ruins e cansaço.

A divisão de tarefas distribui esse peso. Em vez de uma única pessoa carregando tudo, cada membro da equipe carrega apenas sua parte, resultando em menos interrupções, menos retrabalho, menos erros e maior velocidade. Não é apenas porque existem mais pessoas — é porque existe especialização.

III. DIVISÃO DE TRABALHO EM EQUIPES DE SOFTWARE

A. Como Equipes Resolvem o Problema

Imagine uma equipe onde cada pessoa possui uma responsabilidade principal: arquiteto, backend, frontend, QA, DevOps e documentação. Cada profissional mantém seu foco, e o resultado é mais eficiente do que se todos tentassem fazer tudo. A divisão de trabalho não significa apenas dividir pessoas, mas sim dividir responsabilidades.

Uma equipe de dez pessoas em que todos fazem tudo tende a produzir duplicação de trabalho, conflitos de arquitetura, retrabalho e decisões inconsistentes. Já uma equipe onde cada papel está claramente definido — mesmo com o mesmo número de pessoas — tende a ser muito mais eficiente porque reduz ambiguidades e melhora a coordenação.

B. Pipeline de Produção

Empresas grandes trabalham como uma fábrica. Os requisitos passam por arquitetura, que alimenta o desenvolvimento, que é seguido por testes, deploy e monitoramento. Cada etapa possui critérios claros, o que evita que erros avancem para as fases seguintes.

O custo dos erros aumenta exponencialmente quanto mais tarde são detectados. Um bug encontrado durante o planejamento tem custo 1x. Durante o desenvolvimento, 3x. Durante os testes, 10x. Após entrar em produção, 100x. Quanto mais cedo o erro é detectado, menor o custo para corrigi-lo — por isso existem revisões constantes em processos bem estruturados.

C. Paralelismo

Uma pessoa trabalhando sozinha precisa realizar as tarefas sequencialmente: banco,

depois API, depois frontend, depois testes. O tempo total é de 100%. Uma equipe, por outro lado, pode trabalhar em paralelo: banco, API, frontend e testes podem ocorrer simultaneamente. Esse é um dos maiores ganhos de produtividade proporcionados pela divisão de trabalho.

D. Especialização

Na indústria aeronáutica, ninguém espera que um único engenheiro projete sozinho o motor, a fuselagem, a eletrônica, o software embarcado e os sistemas hidráulicos. Cada especialista aprofunda seu conhecimento em um domínio específico. A engenharia de software evoluiu na mesma direção. A especialização reduz o esforço cognitivo individual e aumenta a qualidade das decisões dentro de cada área.

Em 1975, Frederick P. Brooks Jr. publicou a obra *The Mythical Man-Month*, considerada um dos livros mais influentes da engenharia de software. Seu princípio mais conhecido, a Lei de Brooks, afirma: adicionar pessoas a um projeto atrasado tende a atrasá-lo ainda mais. Isso ocorre porque novos integrantes precisam aprender a arquitetura, entender regras de negócio, conhecer padrões internos e comunicar-se com os demais. Brooks mostrou que o número de canais de comunicação cresce aproximadamente pela fórmula $n(n-1)/2$, evidenciando que coordenação também tem custo. Isso não significa que equipes grandes sejam ruins, mas que elas precisam de organização, papéis claros e arquitetura bem definida.

Uma revisão sistemática que analisou 5.414 estudos iniciais e selecionou 102 trabalhos relevantes encontrou que, sob pressão de prazo, muitas equipes produzem mais rapidamente, porém a qualidade do software tende a cair. Esse resultado reforça que produtividade não deve ser medida apenas pela quantidade de código escrito, mas também pela qualidade e sustentabilidade do sistema.

IV. APLICAÇÃO DO PRINCÍPIO À INTELIGÊNCIA ARTIFICIAL

A. O Problema do Prompt Gigante

Tudo o que discutimos até agora se aplica quase perfeitamente aos modelos de linguagem. Muitos usuários constroem um prompt enorme tentando fazer com que um único modelo analise requisitos, projete arquitetura, escreva código, faça testes, documente, revise segurança e otimize desempenho. Na prática, isso equivale a pedir

que um único profissional desempenhe simultaneamente todos os papéis de uma grande equipe.

Imagine um prompt com milhares de linhas. O modelo precisa manter simultaneamente objetivos, regras, contexto, restrições, padrões, exemplos e arquitetura. Isso aumenta a chance de esquecer requisitos, contradizer instruções, misturar responsabilidades e gerar código inconsistente. Mesmo com janelas de contexto grandes, a qualidade do raciocínio pode se degradar quando muitas responsabilidades competem pela atenção do modelo.



Fig. 1. O problema do prompt gigante: um único prompt tenta abranger todas as responsabilidades.

B. Engenharia de Agentes

Em vez de um único agente responsável por tudo, criamos vários agentes especializados, cada um com uma função extremamente clara. Um exemplo típico inclui: Supervisor, Planejador, Executor Backend, Executor Frontend, Executor Banco, Testador e Documentador. Cada agente recebe apenas o contexto necessário para sua função, o que reduz distrações e melhora a qualidade das respostas.

Observe que nenhum agente tenta fazer tudo. Cada um resolve apenas seu problema específico. Esse é exatamente o mesmo princípio da divisão de trabalho em equipes humanas.



Fig. 2. Divisão de problemas complexos em problemas menores e especializados.

C. Fluxo Entre Agentes

O fluxo típico de um sistema multiagente funciona da seguinte forma: o usuário faz uma solicitação ao Supervisor, que a encaminha ao Planejador. Este divide o problema em tarefas e as distribui entre os executores especializados (Backend, Frontend, Banco). Em seguida, um Revisor verifica a consistência, o Testador valida a qualidade e o Documentador prepara a documentação. Por fim, a resposta consolidada é entregue ao usuário.

Cada etapa do fluxo é claramente definida, e nenhum agente precisa conhecer profundamente todas as áreas do sistema. Cada um trabalha com um contexto menor e mais focado.

D. Vantagens dos Subagentes

A abordagem multiagente oferece diversas vantagens. Primeiro, a especialização: cada agente recebe apenas o contexto necessário, o que reduz distrações e melhora a qualidade. Segundo, o contexto reduzido: em vez de um prompt de 100 páginas, cada agente recebe cerca de 10 páginas, aumentando a qualidade e reduzindo alucinações. Terceiro, a reutilização: o agente de testes pode ser usado em qualquer projeto, assim como o de documentação — criam-se componentes reutilizáveis.

Quarto, a facilidade de manutenção: se mudar a documentação, troca-se apenas o documentador; o restante permanece igual. Quinto e mais importante, o paralelismo: vários agentes podem trabalhar ao mesmo tempo — backend, frontend, banco e documentação podem ser processados simultaneamente, reduzindo drasticamente o tempo total.

E. O Orquestrador como Tech Lead

Em equipes humanas existe frequentemente alguém responsável por coordenar o trabalho: um líder técnico, arquiteto ou gerente de engenharia. Nos sistemas de IA, esse papel é desempenhado pelo agente orquestrador. Ele interpreta a solicitação do usuário, divide o problema em subtarefas, encaminha cada subtarefa ao especialista adequado, integra os resultados e verifica a consistência antes da entrega. O orquestrador não precisa ser o melhor programador ou redator; sua principal função é coordenar o fluxo de trabalho.

V. EXEMPLOS PRÁTICOS E ANALOGIAS

A. Método Tradicional vs. Método Moderno

No método tradicional, o usuário envia uma solicitação diretamente a um LLM, que tenta produzir o sistema inteiro em uma única resposta. Esse é o equivalente a um único desenvolvedor tentando fazer tudo sozinho — o resultado costuma ser inconsistente, superficial ou com maior chance de erros.

No método moderno, um orquestrador recebe a solicitação e a encaminha para um planejador, que gera uma lista de tarefas. Essas tarefas são distribuídas para executores especializados (API, Banco, Frontend). Em seguida, um QA valida o resultado, um documentador prepara a documentação e a resposta final é entregue ao usuário. Muito mais organizado e confiável.

B. Analogia com a Construção Civil

Imagine construir um prédio. Você contrataria apenas uma pessoa para desenhar a planta, calcular a estrutura, instalar a parte elétrica, fazer a hidráulica, erguer as paredes, pintar, revisar a qualidade e emitir laudos? Provavelmente não. Existem profissionais especializados: arquiteto, engenheiro estrutural, eletricista, encanador, pedreiro, pintor e inspetor. Cada profissional domina sua área e trabalha em coordenação com os demais.

Um sistema de IA multiagente segue a mesma lógica: um agente orquestrador distribui o trabalho para especialistas, que

depois integram seus resultados. A arquitetura ideal para projetos complexos não é um superagente que faz tudo, mas uma equipe de agentes especializados trabalhando de forma coordenada — assim como uma equipe de engenharia de software bem organizada.

C. Exemplo em Sistemas de Análise de Logs

Em um ambiente com Ollama, OpenCode e modelos locais, uma arquitetura multiagente poderia ser estruturada com um orquestrador que coordena um classificador e um planejador. O classificador alimenta um leitor de logs, enquanto o planejador coordena um especialista Linux. Os resultados fluem para detecção de erros e correlação de eventos, que alimentam um gerador de relatório, passam por um revisor técnico e produzem a resposta final.

Cada agente pode usar um prompt curto, objetivo e altamente especializado, reduzindo a carga de contexto e facilitando a manutenção. Esse paradigma tende a se tornar cada vez mais importante à medida que sistemas baseados em LLMs evoluem em escala e complexidade.

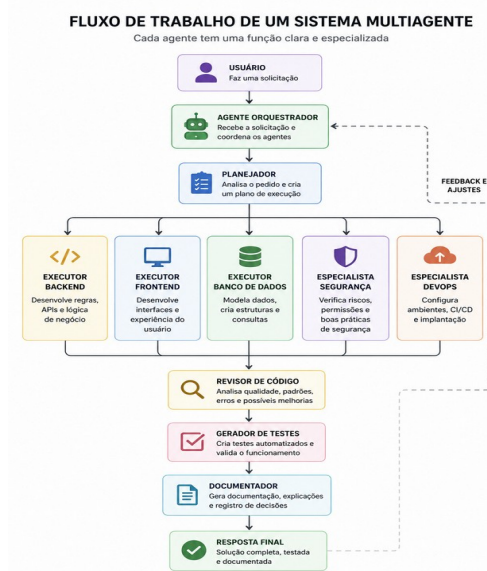


Fig. 3. Fluxo de trabalho em um sistema multiagente com orquestrador e especialistas.

VI. CONCLUSÃO

As evidências da engenharia de software, da psicologia cognitiva e da ciência da computação apontam na mesma direção: sistemas complexos tornam-se mais eficientes quando o trabalho é decomposto em responsabilidades claras. Isso vale para pessoas e, cada vez mais, para inteligência artificial.

Em vez de concentrar toda a responsabilidade em um único desenvolvedor ou em um único prompt, arquiteturas modernas distribuem funções entre especialistas coordenados por um orquestrador. Essa abordagem reduz a carga cognitiva, melhora a qualidade das decisões, diminui a propagação de erros, facilita testes e manutenção e permite evolução contínua do sistema.

No contexto dos LLMs, esse paradigma é particularmente poderoso. Um ecossistema de agentes especializados consegue resolver problemas complexos de maneira mais previsível, auditável e escalável do que um único superprompt. Essa mudança representa uma transição semelhante à que ocorreu na engenharia de software: da figura do programador solitário para equipes organizadas por funções, processos e colaboração baseada em especialização.

A evolução da engenharia de software mostrou que especialização, divisão de responsabilidades e processos bem definidos produzem sistemas de maior qualidade do que concentrar todas as funções em uma única pessoa. A engenharia moderna de IA segue exatamente o mesmo princípio, e essa tendência se consolidará à medida que sistemas baseados em LLMs continuarem a evoluir em escala e complexidade.

REFERÊNCIAS

- F. P. Brooks Jr. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975.
- WikiBooks. *LaTeX/Mathematics*. URL: en.wikibooks.org/wiki/LaTeX/Mathematics (acessado em 15/05/2025).
- P. Lima, E. Guerra, P. Meirelles, L. Kanashiro, H. Silva e F. Silveira. "A Metrics-Driven Approach for Quality Improvement in Software Development." *Journal of Systems and Software*, 2017.
- J. Schell. *The Art of Game Design: A Book of Lenses*. 2nd ed. A. K. Peters, Ltd., 2014.
- P. Lima, E. Guerra, M. Nardes, A. Mocci, G. Bavota e M. Lanza. "An Annotation-based Approach for Detecting Software Defects." In: *Proceedings of the International Conference on Software Engineering*, 2017.
- S. C. Müller e T. Fritz. "Stuck and Frustrated or In Flow and Happy: Sensing Developers' Emotions and Progress." In: *Proceedings of the IEEE/ACM 37th IEEE International Conference on Software Engineering*, 2015.

M. Czerwinski, E. Horvitz e S. Wilhite. "A Diary Study of Task Switching and Interruptions." In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, 2004.

AUTOR

Arthur Delfino Duarte é bacharel em Engenharia Elétrica pelo Instituto Nacional de Telecomunicações (Inatel) e possui sólida experiência em infraestrutura de TI, redes de computadores, data centers e telecomunicações, tendo atuado em empresas como Huawei, Nokia, Net/Claro e Surf Telecom.

Ao longo de sua carreira, participou de projetos de implantação e operação de ambientes críticos, envolvendo sistemas Linux e Windows, roteadores, switches, infraestrutura de data center, monitoramento, recuperação de desastres e serviços de telecomunicações de grande escala. Atualmente, dedica-se ao aprofundamento de seus conhecimentos em Engenharia de Software, Engenharia Backend, DevOps, computação em nuvem, sistemas distribuídos e arquitetura de software, desenvolvendo projetos práticos com Java, Spring Boot, Docker, Kubernetes, APIs REST, bancos de dados relacionais e inteligência artificial executada localmente.

Também possui certificações e treinamentos nas áreas de redes, Linux, computação em nuvem e gerenciamento de projetos, incluindo formações da Red Hat, Microsoft, Google, AWS e certificações profissionais em tecnologias Nokia, Juniper e Microsoft Azure.

Seus principais interesses concentram-se em Engenharia de Software, infraestrutura de alta disponibilidade, confiabilidade de sistemas, observabilidade, inteligência artificial aplicada ao desenvolvimento de software, modelos de linguagem executados localmente (LLMs), automação de infraestrutura e arquiteturas escaláveis para aplicações modernas.